A+B with Binary Search Trees

王文浩

Date: 2026-04-11

Chapter 1：引言

1.1、问题描述

给定两棵二叉搜索树 T1 和 T2，以及一个整数 N。需要从 T1 中找出一个数 A，从 T2 中找出一个数 B，使得 A + B = N。若存在这样的数对，则输出所有不重复的等式（按 A 升序），并输出两棵树的前序遍历序列；否则输出 `false` 并输出前序遍历序列

1.2、算法背景

- **二叉搜索树 (BST)**：一种二叉树，左子树所有节点的值小于根节点，右子树所有节点键值大于等于根节点
- **中序遍历(Inorder Travesal)**：对给定的二叉树，对每个节点，依次遍历**所有左节点**->**根节点**->**所有右节点**，中序遍历的结果是产生**升序序列**
- **前序遍历(Preorder Travesal)**：对给定的二叉树，对每个节点，依次遍历**根节点**->**所有左节点**->**所有右节点**
- **去重**：由于 BST 允许重复值（右子树 ≥ 根），同一个等式可能多次出现，需保证只输出一次

1.3、任务目标

- **建树**：根据输入的节点信息（值、父节点索引）正确构建两棵BST
- **搜索**：找出所有满足 A+B=N 的**不重复**数对。
- **遍历**：正确实现树的遍历，避免最坏情况下（树退化为链表）导致栈溢出。
- **输出**：判断结果（true/false）、输出所有解（按 A 升序），两棵树的前序遍历序列。

1.4、问题分析

- **去重处理**：两个BST中可能含有重复键值，遍历BST_A时需跳过相同 A
- **栈溢出风险**：若使用递归遍历，当树**退化为链表**时（整个树没有左节点或右节点）**递归深度可达 2×10^5**，超出系统栈空间。必须手动维护一个栈实现**迭代遍历**
- **数值范围**：键值范围在 $[-2\times10^9, 2\times10^9]$ 内，加法可能达到 $\pm4\times10^9$，需使用 64 位整数 `long long int` 存储。

Chapter 2：算法规范

2.1、数据结构

二叉树(BST)

每个节点存储三个信息：**value**，**左节点索引**，**右节点索引**。使用数组存储所有节点，通过**root**储存根节点的坐标，通过左右节点的索引访问子节点

```
typedef struct {
    long long int value;       // 节点键值
    int leftIndex;             // 左节点数组索引, -1 表示空
    int rightIndex;            // 右节点数组索引, -1表示空
} Node;

typedef Node *BST;
BST BST_A, BST_B;              // 初始化两个BST
int BST_A_ROOT, BST_B_ROOT;    // 声明两个BST的root
```

栈 (Stack)

用于迭代遍历时模拟系统调用栈。每个栈节点存储一个节点索引，并指向下一个栈节点。

```
typedef struct stack {
    int data;                  // 节点索引
    struct stack *next;        // 下一个节点的索引
} StackNode;
typedef StackNode *head;       //头节点指针
```

2.2、函数设计与实现

2.2.1 建树 (buildBST)

设计思路：
输入给出了每个节点的值及其父节点索引。由于输入给出的顺序是随机的，存储当前元素时候，其父节点可能还是空的，所以无法判断该元素是其父节点的左孩子还是右孩子，且如果一次遍历无法有效初始化**左/右节点索引**，因此，我们需要两次遍历：
第一次遍历：读取所有节点值到数组中，并初始化**左右节点索引**为 -1
第二次遍历：根据**父节点索引**和**BST**性质为每个父节点设置左/右孩子。若父节点值为 -1 则该节点为根节点。

伪代码：

```
函数 buildBST(ptrBST, size)
输入: 节点数组指针, 节点个数 size
输出: 根节点索引

如果 size == 0 则返回 -1
分配 Parent 数组, 长度为 size
对于 i = 0 到 size-1:
    读入 value 和 parent
    ptrBST[i].value = value
    ptrBST[i].leftIndex = -1
    ptrBST[i].rightIndex = -1
    Parent[i] = parent
对于 i = 0 到 size-1:
    如果 Parent[i] == -1:
        rootIndex = i
    否则如果 ptrBST[Parent[i]].value > ptrBST[i].value:
        ptrBST[Parent[i]].leftIndex = i
    否则:
        ptrBST[Parent[i]].rightIndex = i
释放 Parent
返回 rootIndex
```

代码实现

```c
int buildBST(BST ptrBST, int size) {
    if (size <= 0) {
        return -1; /*root = -1 mean the tree is empty*/
    }
    int rootIndex = -1;
    int value;
    int *Parent = malloc(sizeof(int) * size);

    // First read the value and the parent of each node
    for (int i = 0; i < size; i++) {
        scanf("%d %d", &value, &Parent[i]);
        ptrBST[i].value = value;   /*Store the value of the node*/
        ptrBST[i].leftIndex = -1;  /*Initialize the left index*/
        ptrBST[i].rightIndex = -1; /*Initialize the right index*/
    }

    // Second loop: build the tree according to the parent array
    for (int i = 0; i < size; i++) {
        if (Parent[i] == -1) {
            rootIndex = i; /*Store the index of the root node*/
        }
        else if (ptrBST[Parent[i]].value > ptrBST[i].value) { /*smaller than parent*/
            ptrBST[Parent[i]].leftIndex = i;
        }
        else {
            ptrBST[Parent[i]].rightIndex = i;
        }
    }
    free(Parent);
    return rootIndex; /*Return the index of the root node*/
}
```

2.2.2 前序遍历 (preorder_prt)

设计思路:

由于数据范围可达到**$2x10^5$**,直接使用**递归遍历**可能会有爆栈风险(当树退化为链表时，递归深度为$2x10^5$)，所以使用栈模拟递归实现前序遍历，遍历顺序为**根->左节点->右节点**，步骤如下:

- **初始化**，将根节点压入栈
- **循环**，弹出栈顶，然后压入子节点，由于栈是后进先出，需先将右孩子压栈，再压左孩子，保证左孩子先被弹出。所有栈节点动态分配并在使用后释放。确保空间复杂可控，避免爆栈风险。
- **结束**：循环直到栈为空，遍历完所有节点

伪代码：

```
函数 preorder_prt(ptrBST, root)
输入: 节点数组, 根索引
输出: 无（打印前序序列, 末尾换行）

如果 root == -1:
    打印换行
    返回
初始化栈为空
创建新栈节点, data = root, 压栈
flagSpace = 0
当栈非空:
    弹出栈顶节点, cur = 节点.data
    如果 flagSpace == 0:
        打印 ptrBST[cur].value
        flagSpace = 1
    否则:
        打印 " " + ptrBST[cur].value
    释放弹出节点
    如果 ptrBST[cur].rightIndex != -1:
        创建新栈节点, data = rightIndex, 压栈
    如果 ptrBST[cur].leftIndex != -1:
        创建新栈节点, data = leftIndex, 压栈
打印换行
```

```c
void preorder_prt(BST ptrBST, int root) {
    if (root == -1) {
        return; /*return when the size of the BST is 0*/
    }
    int flagSpace = 0;
    // Initial the stack for the preorder travesal
    head stack = NULL;
    StackNode *newNode = malloc(sizeof(StackNode));
    newNode->data = root;
    newNode->next = NULL;
    stack = newNode;
    StackNode *currentNode = stack; /*point to the top of the stack*/

    while (stack != NULL) {  /*Run the loop until the stack is empty*/
        currentNode = stack;
        stack = stack->next;
        // Print the value of the top of the stack
        int current = currentNode->data;
        if (flagSpace == 0) {
            printf("%lld", ptrBST[current].value);
            flagSpace = 1; /*Update the flag*/
        }
        else {
            printf(" %lld", ptrBST[current].value);
        }
        free(currentNode);
        // Push the right child of the top node into the stack if it exists
        if (ptrBST[current].rightIndex != -1) {
            StackNode *newNode = malloc(sizeof(StackNode));
            newNode->data = ptrBST[current].rightIndex;
            newNode->next = stack;
            stack = newNode;
        }
        // Push the left child of the top node into the stack if it exists
        if (ptrBST[current].leftIndex != -1) {
            StackNode *newNode = malloc(sizeof(StackNode));
            newNode->data = ptrBST[current].leftIndex;
            newNode->next = stack;
            stack = newNode;
```

```
            }
        }
        printf("\n");
    }
```

2.2.3 构建搜索数组 (buildSearchArray)

设计思路：

1. **需要SearchArray的原因**：如果BST_B退化成链表结构，那对BST_B进行**二分查找**就会退化成**顺序查找**，**时间复杂度达到O(N)**，所以，通过一次遍历，将BST_B的元素通过升序存入一个用于查找BST_B中元素的数组，对该数组进行查找可以确保**时间复杂度为O(logN)**，综上所诉，对BST_B进行中序遍历，得到升序序列。由于可能重复值，遍历时跳过连续相同的值，只将不重复的键值存入数组。该数组将用于二分查找。
2. **中序遍历**：遍历顺序为**左节点->根->右节点**，步骤如下：
    - **循环**压入左节点直到左节点不存在，弹出栈顶，进入右子树继续上述循环。所有栈节点动态分配并在使用后释放。确保空间复杂可控，避免爆栈风险。
    - **结束**：循环直到栈为空，遍历完所有节点

伪代码：

```
函数 buildSearchArray(ptrBST, root, size)
输入: 节点数组, 根索引, 最大可能长度
输出: 动态分配的去重升序数组, 实际长度存入全局 ArraySize

如果 root == -1:
    ArraySize = 0
    返回 NULL
分配 searchArray, 容量 size
index = 0, lastOut = EMPTY
初始化栈为空
current = root
当 current != -1 或栈非空:
    当 current != -1:
        将 current 压栈
        current = ptrBST[current].leftIndex
    弹出栈顶到 current
    如果 ptrBST[current].value != lastOut:
        searchArray[index++] = ptrBST[current].value
        lastOut = ptrBST[current].value
    current = ptrBST[current].rightIndex
ArraySize = index
返回 searchArray
```

代码实现:

```c
long long int *buildSearchArray(BST ptrBST, int root, int size) {
    if (root == -1) { /*If BST is empty, return NULL*/
        return NULL;
    }

    // Initial the search array
    long long int *searchArray = malloc(sizeof(long long int) * size);
    int index = 0;                 /*Initial index*/
    long long int lastOut = EMPTY;
    // inorder iterator
    // Initial the stack for the inorder travesal
    head stack = NULL;
    int current = root;
    StackNode *currentNode;
    while (current != -1 || stack != NULL) {
        while (current != -1) {
            StackNode *newNode = malloc(sizeof(StackNode));
            newNode->data = current;
            newNode->next = stack;
            stack = newNode;
            current = ptrBST[current].leftIndex;
        }

        // Pop the top of stack
        currentNode = stack;
        stack = stack->next;

        // Store the value of the top of the stack into searcharry
        current = currentNode->data;
        if (ptrBST[current].value != lastOut) {
            searchArray[index++] = ptrBST[current].value;
            lastOut = ptrBST[current].value;
        }
        free(currentNode);
        // turn to right child tree
        current = ptrBST[current].rightIndex;
    }
    // Store the number of different elements
    ArraySize = index;
```

```
        return searchArray; /*Return the pointer to the search array*/
    }
```

2.2.4 二分查找 (search)

设计思路:
在有序、无重复的数组上执行标准二分查找。每次取中间元素比较，缩小一半区间，直到找到目标或区间为负值或空。

伪代码:

```
函数 search(arr, size, target)
输入: 升序数组, 数组长度, 目标值
输出: 1 如果找到, 否则 0

left = 0, right = size - 1
当 left <= right:
    mid = (left + right) / 2
    如果 arr[mid] == target:
        返回 1
    否则如果 arr[mid] < target:
        left = mid + 1
    否则:
        right = mid - 1
返回 0
```

代码实现:

```
int search(long long int *searchArray, int size, long long int value) {
    int left = 0, right = size - 1;
    while (left <= right) {
        int mid = (left + right) / 2;    /*Calculate the middle index*/
        if (searchArray[mid] == value) { /*key found*/
            return 1;
        }
        else if (searchArray[mid] < value) { /*key lies in the left half*/
            left = mid + 1;
        }
        else { /*key lies in the right half*/
            right = mid - 1;
        }
    }
    return 0; /*key not found*/
    }
```

2.2.5 中序遍历查找 (inorder_iterator)

设计思路:
对**BST_A**进行**迭代中序遍历**，依次得到**升序的BST_A**的值。（利用 `lastout` 跳过重复的 A）对于每个不重复的 A，计算 `complement = N - A`，然后调用 `search` 在通过**BST_B生成的searchArray**中查找 complement。若找到，则将该 A 存入答案数组。由于中序遍历结果是升序的，答案数组**自动按 A 升序排列**且**无重复值**。

伪代码:

```
函数 inorder_iterator(ptrBST, root, N, searchArray)
输入: T1 节点数组, 根索引, 目标值 N, T2 去重升序数组
输出: 无（将找到的 A 存入全局 answer, answerCount 递增）

如果 root == -1: 返回
初始化栈为空
current = root
lastout = EMPTY
当 current != -1 或栈非空:
    当 current != -1:
        将 current 压栈
        current = ptrBST[current].leftIndex
    弹出栈顶到 current
    如果 ptrBST[current].value != lastout:
        complement = N - ptrBST[current].value
        如果 search(searchArray, ArraySize, complement) == 1:
            answer[answerCount++] = ptrBST[current].value
        lastout = ptrBST[current].value
    current = ptrBST[current].rightIndex
```

代码实现:

```
void inorder_iterator(BST ptrBST, int root, long long int key, long long int *searchA
    if (root == -1) { /*Return if the BST is empty*/
        return;
    }
    // Avoid the duplicate value in the BST_A,
    //use the variable lastout to store the last value
    // output from the stack, and ignore the value when it's same to latout
    long long int lastout = EMPTY; /*Initialize the lastout variable*/

    // Initial the stack for the inorder travesal
    head stack = NULL;
    int current = root;
    StackNode *currentNode;

    while (current != -1 || stack != NULL) {
        while (current != -1) { /*Run until the node have no left child*/
            /*store the current node in the stack*/
            StackNode *newNode = malloc(sizeof(StackNode));
            newNode->data = current;
            newNode->next = stack;
            stack = newNode;
            current = ptrBST[current].leftIndex;
        }

        // Pop the top of stack
        currentNode = stack;
        stack = stack->next;

        // check value of the top of the stack,
        //and search complementary value in the searchArray
        current = currentNode->data;
        if (ptrBST[current].value != lastout) { /* check if it not a duplicate value*/
            long long int complement = key - ptrBST[current].value;
            if (search(searchArray, ArraySize,
                    complement)) { /*If the complementary value is found*/
                answer[answerCount++] =
                    ptrBST[current].value; /*store the value in the answer array*/
            }
            lastout = ptrBST[current].value; /*Update the lastout variable*/
```

```
            }
            free(currentNode);
            current = ptrBST[current].rightIndex; /*Turn to the right child*/
        }
    }
```

Chapter 3：测试结果

3.1、测试样例与h具体分析

测试样例文件	测试目的
test1.txt	验证题目给出的标准样例，确保基本功能正确
test2.txt	验证题目给出的标准样例，覆盖无解情况和中序遍历输出
test3.txt	边界情况：两棵 BST 均为空树，检查空树处理的健壮性
test4.txt	存在重复解时，验证程序是否正确去重
test5.txt	数据包含负值，验证负值情况下的求解与输出正确性
test6.txt	数据值接近整型边界，验证边界值条件下的求解正确性
test7.txt	目标和超出树中最大元素值，验证极限搜索的正确性
test_20.txt ~ test_2000000.txt	不同数据规模（从极小到上限）的压力测试，验证程序在大数据量下的运行稳定性

测试样例1：

test1.txt：

```
8
12 2
16 5
13 4
18 5
15 -1
17 4
14 2
18 3
7
20 -1
16 0
25 0
13 1
18 1
21 2
28 2
3
```

预期结果：

```
true
36 = 15 + 21
36 = 16 + 20
36 = 18 + 18
15 13 12 14 17 16 18 18
20 16 13 18 25 21 28
```

测试样例说明：与题目给出的测试样例一致

测试样例2：

test2.txt：

```
5
10 -1
5 0
15 0
2 1
7 1
3
15 -1
10 0
20 0
4
```

预期结果：

```
false
10 5 2 7 15
15 10 20
```

测试样例说明：与题目给出的测试样例一致

测试样例3：

test3.txt：

```
0
0
1
```

预期结果：

```
false
```

测试样例说明：边界情况：BST_A和BST_B都为空，确保在BST为空时正常运行

测试样例4：

test4.txt：

```
5
10 -1
5 0
10 0
3 1
10 2
3
10 -1
10 0
10 1
20
```

预期结果：

```
true
20 = 10 + 10
10 5 3 10 10
10 10 10
```

测试样例说明：有重复解，检测程序是否做到去重

测试样例5：

test5.txt：

```
2
0 -1
-5 0
2
0 -1
5 0
0
```

预期结果：

```
true
0 = -5 + 5
0 = 0 + 0
0 -5
0 5
```

测试样例说明：检测负值数据是否能正确求解

测试样例6：

test6.txt：

```
3
2000000000 -1
1999999999 0
1999999998 1
3
1 -1
2 0
3 1
2000000001
```

预期结果：

```
true
2000000001 = 1999999998 + 3
2000000001 = 1999999999 + 2
2000000001 = 2000000000 + 1
2000000000 1999999999 1999999998
1 2 3
```

测试样例说明：检测当数据大小趋近边界时是否可以正常求解

test7.txt：

```
3
2000000000 -1
1999999999 0
1999999998 1
3
2000000000 -1
1999999999 0
1999999998 1
4000000000
```

预期结果：

```
true
4000000000 = 2000000000 + 2000000000
2000000000 1999999999 1999999998
2000000000 1999999999 1999999998
```

测试样例说明：检测当搜索值大于BST中数据上限值是否可以正常求解

剩余测试样例

test_20.txt ~ test_2000000.txt

注：该组样例由脚本生成，用于测试不同数据规模情况下的代码运行情况，仅作生成大规模数据使用，仅供参考

测试样例说明：生成不同数据规模的测试数据，检测程序在数据量达到数据规模上限时是否正常运行

Chapter 4：复杂度分析

4.1、各函数复杂度分析

4.1.1、建树(buildBST)

1. **时间复杂度O(N)**：**遍历两遍数组**，第一遍存储数据和初始化左右子节点，第二遍修改左右子节点指向关系，则时间复杂度为**O(N)**
2. **空间复杂度O(N)**：需要建立一个大小为N的数组，空间复杂度为**O(N)**

4.1.2、前序遍历(preorder_prt)

1. **时间复杂度O(N)**:每个节点只访问一遍，共需要访问N次，则时间复杂度为**O(N)**
2. **空间复杂度O(h)/O(1)**: 空间复杂度取决与树的形状，当树退化为链表时，栈每次存入一个，弹出一个，空间复杂度为**O(1)**，若树为平衡数，则空间复杂度取决与最深的路径上有多少个右节点，即**O(h)/O(log N)**,最坏情况可达到**O(N)**(当每个节点都有左节点，且右节点仅有一个，这样，该栈可能会把所有右节点存进栈而没有弹出),

4.1.3、构建搜索数组(buildSearchArray)

1. **时间复杂度O(N)**:通过中序遍历，每个节点只访问一遍，共需要访问N次，则时间复杂度为**O(N)**
2. **空间复杂度O(N)**:需要建立一个大小为N的Search Array，空间复杂度为**O(N)**

4.1.4、二分查找(search)

1. **时间复杂度O(N)**:通过二分查找，每次访问缩小一半的区间，最多访问(logN + 1)次，则时间复杂度为**O(log N)**
2. **空间复杂度O(1)**:迭代版本的二分查找，不需要额外空间

4.1.5、构建搜索数组(buildSearchArray)

1. **时间复杂度O(N log M)**:通过中序遍历BST_A，每个节点只访问一遍，共需要访问N次，对每个A值，都要对Search Array进行二分查找，搜索N-A是否存在，搜索需要O(log M)，则总的时间复杂度为N*logM 为**O(N log M)**

2. **空间复杂度O(h)/O(1)**:该函数使用了中序迭代，需计算栈的最大深度。栈的深度取决于最多同时存入多少个左节点，当数组退化为链表时，若只有左子树，空间复杂度为**O(N)**，若只有右子树，空间复杂度为**O(1)**。若较为平衡，则最多同时存入多少左节点取决于树的高度，空间复杂度为**O(h)**，平均值为**O(log N)**

4.2、总结

函数	时间复杂度	空间复杂度	说明
`buildBST`	O(N)	O(N)	遍历所有节点两次，额外分配 Parent 数组
`preorder_prt`	O(N)	平均O(h)	每个节点访问一次，栈空间等于树高
`buildSearchArray`	O(M)	O(M)	中序遍历每个节点一次，额外分配去重数组
`search`	O(log M)	O(1)	二分查找，无须额外空间
`inorder_iterator`	O(N log M)	平均O(h)	遍历 BST_A 每个节点，对每个不重复 A 进行 O(log M) 的二分查找

> h 为树高，平均为log N。最坏情况下树退化为链表，h = N 或 M，但迭代栈使用堆内存，不会导致系统栈溢出。

4.3、总体复杂度

- **时间复杂度**:
  构建两棵树 O(N + M);
  构建 BST_B 的搜索数组 O(M);
  遍历 BST_A 并二分查找 O(N log M)。
  因此总时间复杂度为**O(N log M)**，最坏情况也为**O(N log M)**
- **空间复杂度**:
  存储两棵树节点 O(N + M);
  存储 BST_B 搜索数组 O(M);
  存储答案 O(N);
  栈空间 O(h)（堆分配，最后全部释放）。
  总体空间复杂度为 **O(N + M)**。

4.4、反思与改进

- **稳定性**：本算法不依赖于树的平衡性，即使树退化为链表，算法皆可正常工作，时间复杂度保持 O(N log M)。
- **时间优化**：可使用哈希集合将 BST_B 的值存入，使查找时间复杂度降为**O(1)**，但需处理值域过大问题（±2e9），空间复杂度剧增。二分查找实现简单且可靠
- **优化方案**：使用类似双指针法的思路，实现 **O(N)** 的时间复杂度，具体操作为，将指针初始化为B数组最后一个元素，对A遍历的同时向前移动指针，直到找到对应的解B，或是`A~n~ + B~n~ < N`，由于对A的遍历，则下一个A_{n+1}对应的B_{n+1}一定比B_n小，则一定在指针指向元素及之前出现，当指针遍历完B数组，或是遍历完B数组，即可以停止查找，时间复杂度为**O(min(n1,n2))**

声明

我在此声明项目“*A+B with Binary Search Trees*”里的所有内容均由我自己独立完成。

I hereby declare that all the work done in this project titled "*A+B with Binary Search Trees*" is of my independent effort.